

WEEK 9 — MITIGATION & HARDENING

Patch Verification, Adjacent Failures, Property Tests, Updated Threat Model

Team ZeroDay | Applied Cryptography | Spring 2026

Work Window: Apr 26 – May 2 | Due: Sun, May 3, 2026 | Patches: ZD-2026-001 ... 005

1. Executive Summary

This week we shipped the mitigation for the Week 8 vulnerability (ZD-2026-001 — revocation bypass via signature replay) and hardened the system against four adjacent failure modes discovered during root-cause review. We rebuilt the original exploit, reran it against the patched server, and captured evidence that it now fails. We added 33 new security tests (9 Hypothesis property tests + 24 negative/fuzz tests) that exercise all the hardening, and updated both the Threat Model and CDDR documents to reflect the final design.

1.1 Patches Shipped

ID	Title	Severity	Status	Where
ZD-2026-001	Revocation Bypass via Sig Replay	HIGH	Fixed (W8)	main.py: register_key
ZD-2026-002	Doc Signature Replay	MEDIUM	Fixed (W9)	main.py: log_document
ZD-2026-003	Semantic Chain Validation Gap	MEDIUM	Fixed (W9)	blockchain.py: validate_chain
ZD-2026-004	Bulk Username Squatting (DoS)	MEDIUM	Fixed (W9)	main.py: RateLimiter
ZD-2026-005	Loose Input Validation	LOW	Fixed (W9)	main.py: pydantic models

2. Fix Correctness (4 pts)

Rubric: Mitigates root cause without breaking MVP.

2.1 ZD-2026-001 — Root Cause Fix

The Week 8 root cause was that the registration endpoint did not check whether a public key had ever been revoked, so a captured registration signature could be replayed indefinitely after the user revoked their key. The fix:

```
# server/app/main.py — register_key handler
@app.post("/api/register")
async def register_key(req: RegisterRequest, _: None = Depends(rate_limit_mutation)):
    # Reject if user_id already has an active key
    existing = bc.lookup_key(req.user_id)
    if existing:
        raise HTTPException(409, ...)

    # ZD-2026-001 fix: reject any public_key that has ever been revoked.
    if bc.is_key_revoked(req.public_key):
        log.warning(f"REGISTER denied: public_key has been revoked previously")
        raise HTTPException(409, "This public key has been revoked and cannot "
                                "be re-registered. Generate a new key pair.")
    # ... (signature verification continues as before)
```

Why this is correct: The check rejects the specific replay vector demonstrated by the exploit (registration of a previously revoked key). It is implemented at the chain layer using a SQLite query that scans all REVOKE blocks for a matching public key — not a cached set, so it cannot become stale. The rejection happens BEFORE signature verification, so an attacker cannot bypass it by manipulating the signature.

Why MVP is not broken: Legitimate users never re-register a revoked key (they generate a new key pair instead). The 12 endpoint tests in test_endpoints.py and the WebSocket signaling test still pass, confirming all happy-path flows are unaffected.

2.2 MVP Compatibility Verification

```
$ python server/test_endpoints.py
=====
ALL TESTS PASSED (12/12)
=====

$ python server/test_signaling.py
=====
WEBSOCKET SIGNALING TEST PASSED
=====
```

3. Verification: Attack Rerun (2 pts)

Rubric: Attack rerun fails + evidence included.

3.1 Methodology

- **Same exploit script:** security/exploits/exploit_revocation_bypass.py is the unmodified Week 8 attack tool. No changes were made to it.
- **Patched server:** The Week 9 server build with all five patches applied.
- **Fresh database:** server/data/blockchain.db deleted before each run for reproducibility.

3.2 Result

```
$ python security/exploits/exploit_revocation_bypass.py

[1] Alice registers: status=200
[2] Alice revokes: status=200
[7] Lookup confirms revocation: status=404

[8] Eve replays the original registration payload byte-for-byte
    x Replay rejected: 409 {"detail":"This public key has been revoked
      and cannot be re-registered. Generate a new key pair."}

    Vulnerability is NOT present in this build.
```

Compare with Week 8 unpatched run: 51 bogus REGISTER blocks accepted.

3.3 Chain State Comparison

Metric	Before fix (Week 8)	After fix (Week 9)
Total blocks	53	3
REGISTER blocks	52 (1 legitimate + 51 bogus)	1
REVOKE blocks	1	1
GENESIS	1	1
Replay attempts succeeded	51	0
Chain integrity check	valid: true (false sense of security)	valid: true (correctly)

3.4 Evidence Files

File	Contents
security/evidence/post_fix/exploit_run_after_fix.txt	Full stdout of the Week 8 exploit script run against the patched server. Ends with "Vulnerability is NOT present in this build."
security/evidence/post_fix/server_after_fix.log	Server-side log captured during the failed exploit. Shows REGISTER ok, REVOKE ok, REGISTER denied (warning).
security/evidence/post_fix/chain_after_fix.json	Full chain dump after attack: 3 blocks total (genesis + 1 register + 1 revoke).
security/evidence/post_fix/stats_after_fix.json	Final chain stats: REGISTER=1, REVOKE=1, no inflation.

4. Hardening: Adjacent Failures (2 pts)

Rubric: Covers adjacent cases (replay windows, nonce discipline, validation).

Root-cause analysis of ZD-2026-001 surfaced a class of “replayable signed message” issues. We audited the four other signed-message formats (revoke, document, handshake-init, handshake-resp) and the input-validation surface across all endpoints. Four adjacent issues were found and patched.

4.1 ZD-2026-002 — Document Signature Replay

- **Vector:** The document-signature endpoint accepted unlimited duplicate (signer_id, doc_hash, signature) tuples. Each replay added a new chain block. A single legitimate signature could be replayed forever to inflate the chain.
- **Fix:** New helper Blockchain.doc_signature_exists(signer_id, doc_hash) queries SQLite for any prior DOC_SIGNATURE block with matching (signer, hash). Endpoint returns HTTP 409 on duplicate.
- **Verified by:** test_n4_duplicate_doc_signature_rejected; PROP-8 (test_doc_signature_duplicate_detection)

4.2 ZD-2026-003 — Semantic Chain Validation Gap

- **Vector:** validate_chain() only checked hash linkage. A block whose payload was missing required fields (e.g., a REVOKE without a public_key) still passed validation. An attacker who briefly compromised the server's DB file could insert malformed blocks and the chain would still report valid: true.
- **Fix:** PAYLOAD_SCHEMA dict declares required fields per block type. validate_chain() now also verifies (a) payload conforms to schema, (b) block indices are sequential from 0, (c) genesis is at index 0. Backward-compatible flag: validate_chain(semantic=False) for diagnostic use.
- **Verified by:** PROP-1 (validate_chain after random sequences); PROP-5 (sequential indices); PROP-6 (tampering invalidates chain)

4.3 ZD-2026-004 — Bulk Username Squatting / Storage DoS

- **Vector:** With no rate limit, an attacker could script tens of thousands of registrations from a single IP, exhausting server disk and slowing chain queries (linear scans become O(n) over a polluted chain).
- **Fix:** In-memory sliding-window RateLimiter: 30 mutating operations per 60 seconds per IP. Applied as a FastAPI Depends() to /api/register, /api/revoke, and /api/document. Honors X-Forwarded-For for deployments behind Vercel/Railway.
- **Verified by:** test_f3_rate_limit_eventually_kicks_in

4.4 ZD-2026-005 — Loose Input Validation

- **Vector:** RevokeRequest and DocumentSignatureRequest accepted strings of arbitrary length and format. Pydantic only checked field presence. Long inputs reached crypto verification before being rejected.
- **Fix:** All hex fields (public_key, doc_hash) constrained to exactly 64 lowercase hex chars; signatures to exactly 128. user_id and signer_id constrained to `^[a-zA-Z0-9_-]{1,64}$`. Pydantic rejects with 422 before any handler runs.
- **Verified by:** Negative tests N1-N7 (eight parametrized cases each); F1 fuzz test (50 random inputs)

4.5 Hardening Audit Matrix

Concern	Pre-W9	Post-W9	Mechanism
Replay (registration)	Vulnerable	Fixed	is_key_revoked check
Replay (document)	Vulnerable	Fixed	doc_signature_exists check
Replay (handshake)	N/A	N/A	Already protected by ephemeral keys
Replay (chat msg)	N/A	Protected	Monotonic seq numbers (pre-existing)
Nonce uniqueness	Protected	Protected	base XOR seq (10k uniqueness test)
Input validation	Partial	Strict	Pydantic regex + length bounds
Rate limiting	None	30/min/IP	RateLimiter dependency
Chain integrity	Hash linkage only	+ payload schema	validate_chain semantic mode
Path traversal in IDs	Vulnerable	Blocked	Username regex

5. Expanded Security Test Suite

The security/regression_tests/ folder now contains **35 tests** across three modules. All pass on the patched build.

5.1 Property Tests (test_property_chain.py — 9 tests)

Hypothesis-based property tests generate hundreds of randomized inputs to verify invariants hold for ALL valid op sequences, not just hand-picked examples.

ID	Property	Why It Matters
PROP-1	validate_chain() after random ops	If randomly-chosen valid operations break validation, the schema is too strict.
PROP-2	Revoked user never resolves	Ensures the W8 fix holds across input space, not just the specific exploit case.
PROP-3	Lookup returns latest unrevoked key	Catches bugs in the multi-revoke / multi-register sequence.
PROP-4	list_users excludes revoked	Catches inconsistencies between lookup_key and list_users.
PROP-5	Block indices sequential from 0	Required for ZD-2026-003 fix.
PROP-6	Tampering invalidates chain	Verifies validate_chain catches every mutation.
PROP-7	Block hashing deterministic	Required for chain reproducibility.
PROP-8	doc_signature_exists isolates by signer/hash	Required for ZD-2026-002 fix — different signer or hash should not match.
PROP-9	Register-revoke cycles preserve validity	Combination of PROP-1, 2, 5.

5.2 Negative & Fuzz Tests (test_negative_endpoints.py — 24 tests)

Real HTTP requests against a fresh server, exercising the rejection paths.

ID	Cases	Coverage
N-1	3	Public key wrong length, too long, non-hex → 422
N-2	8	Username with whitespace, slash, @, path traversal, null byte, etc. → 422
N-3	1	Signature shorter than 128 chars → 422
N-4	1	Duplicate doc signature → 409 (ZD-2026-002)
N-5	1	Cross-key signature (sign with B, claim A's key) → 400
N-6	2	Empty body, missing fields → 422
N-7	4	Lookup with path traversal payloads → not 5xx, not 200
N-8	1	Lookup of unknown user → 404 (not 500)
F-1	50	Hypothesis-fuzzed register with arbitrary string fields → never 5xx
F-2	30	Hypothesis-fuzzed doc_hash → only 64-char hex makes it past validation
F-3	1	Rate limit triggers within 50 sequential requests

5.3 Test Run

```
$ python -m pytest security/regression_tests/ -v
.....S..... [100%]
===== 35 passed in 15.21s =====

test_property_chain.py          9 tests  (Hypothesis property-based)
test_negative_endpoints.py      24 tests  (Negative + fuzz)
test_revocation_bypass.py       2 tests  (W8 baseline regression)
```

6. Documentation Updates (2 pts)

Rubric: Threat model + CDDR reflect final design.

6.1 Threat Model (docs/THREAT_MODEL.md)

Version bumped from 1.0 to 1.1. Three new threat entries (T9, T10, T11) and four new security requirements (SR-11 through SR-14). T7 (Key Compromise) status updated to reflect the ZD-2026-001 mitigation.

- **T9 — Document signature replay:** new threat. Mitigated by ZD-2026-002 fix.
- **T10 — Bulk registration / storage DoS:** new threat. Mitigated by ZD-2026-004 rate limiting.
- **T11 — Semantic chain corruption:** new threat. Mitigated by ZD-2026-003 fix.
- **SR-11 through SR-14:** new requirements with explicit verification mappings.
- **T7 status:** updated to "Mitigated" with reference to ZD-2026-001.

6.2 CDDR (docs/CDDR.md)

Version bumped from 1.0 to 1.1. New Section 6 documents the five Week 9 patches in concise tabular form. Section 5 (Signed-Message Formats) updated with a 'Lessons from ZD-2026-001' subsection that explains why domain-prefixing is necessary but not sufficient — a candidate v1.1 enhancement (binding signed messages to chain tip) is documented as future work.

6.3 API Reference (docs/API.md)

/api/register error table now includes 409 (revoked key) and 429 (rate limited).

/api/document error table now includes 409 (duplicate signature) and 429.

6.4 PR Description (security/MITIGATION_PR.md)

A standard PR description with summary table, file-change list, verification logs, and reviewer assignments — the document that would accompany the actual GitHub pull request.

7. Files Changed This Week

File	Change	Purpose
server/app/main.py	+49 / -7	Rate limiter, input validation tightening, ZD-2026-001 + ZD-2026-002 fixes
server/app/blockchain.py	+60 / -8	doc_signature_exists, semantic validate_chain, PAYLOAD_SCHEMA
security/regression_tests/test_property_chain.py	NEW (~270 LOC)	9 Hypothesis property tests
security/regression_tests/test_negative_endpoints.py	NEW (~330 LOC)	24 negative + fuzz tests
security/MITIGATION_PR.md	NEW	Standard PR description
security/evidence/post_fix/	NEW (4 files)	Evidence the exploit now fails
docs/THREAT_MODEL.md	NEW (v1.1)	T9, T10, T11; SR-11 through SR-14
docs/CDDR.md	NEW (v1.1)	Section 6 (hardening); ZD lessons
docs/API.md	UPDATED	Error tables include 409, 422, 429
pytest.ini	NEW	Register the security marker

8. Lessons Learned

- **One vulnerability is rarely alone.** The root cause of ZD-2026-001 (replayable static-message signatures) is a pattern, not an isolated bug. The audit revealed the same pattern in document signing (ZD-2026-002) which the original report flagged as low-impact but is in fact a chain-growth DoS.
- **Property tests catch state bugs that example tests miss.** While writing PROP-3 (lookup returns latest unrevoked key), Hypothesis quickly produced a register-revoke-register-new sequence that we hadn't included in any unit test — forcing us to confirm the iteration logic in lookup_key handles it correctly.
- **Validation is layered defense.** Tightening pydantic validation does not stop crypto-level attacks, but it dramatically reduces the surface area where bugs can hide. After ZD-2026-005, malformed inputs are rejected by the framework before any handler logic runs — making it much harder for fuzzing to find anything new.
- **Chain integrity ≠ chain truthfulness.** The original validate_chain(), like many naive blockchain implementations, checked that no one tampered with stored bytes — but said nothing about whether the bytes themselves form a meaningful audit trail. Semantic validation (PAYLOAD_SCHEMA) closes this gap.

- **Rate limits are surprisingly easy to add.** Two dozen lines of code (RateLimiter class + Depends wiring) prevent a class of DoS attacks. Worth doing on every new mutating endpoint, not as an afterthought.